

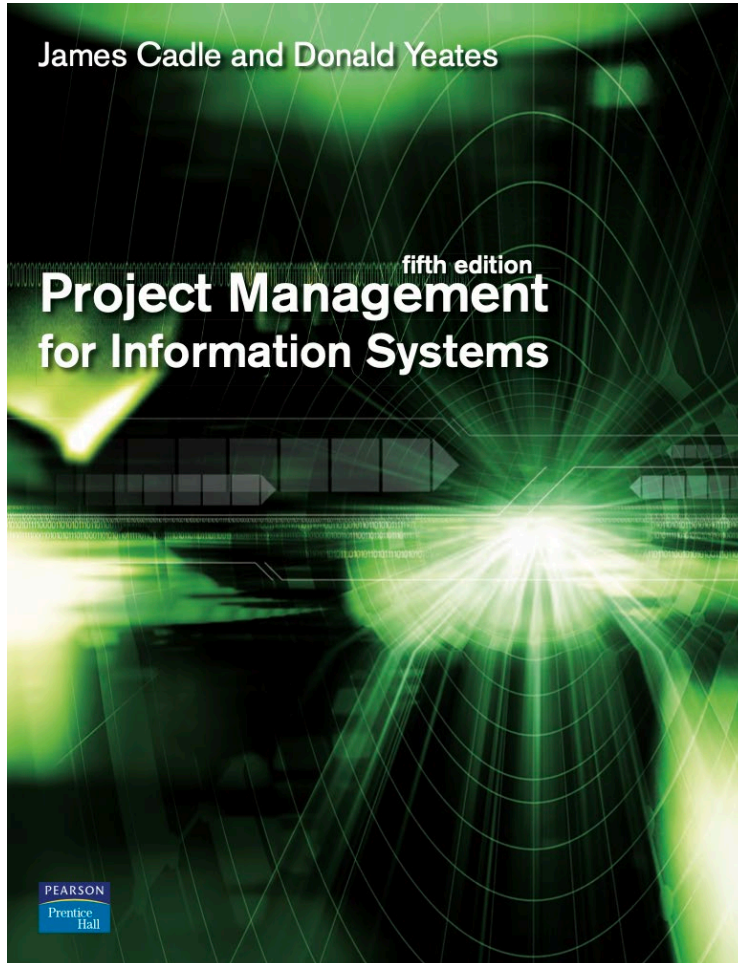
BU5300 – IT Project Management

Lecture 6: Requirements Management and Effort Estimation

Chapters 9 and 10

Overview of the Course

Lectures and Book Chapters



Lecture 1 - Chapters 1 and 2

Lecture 2 - Chapters 3 and 4

Lecture 3 - Chapter 6

Lecture 4 - *Agile Development*

Lecture 5 - Chapters 7 and 8

Lecture 6 - Chapter 9 and 10

Lecture 7 - Chapters 11, 12 and 13

Lecture 8 - Chapters 14 and 15

Lecture 9 - Chapters 18 and 21

Lecture 10 - Chapters 20, 22 and 23

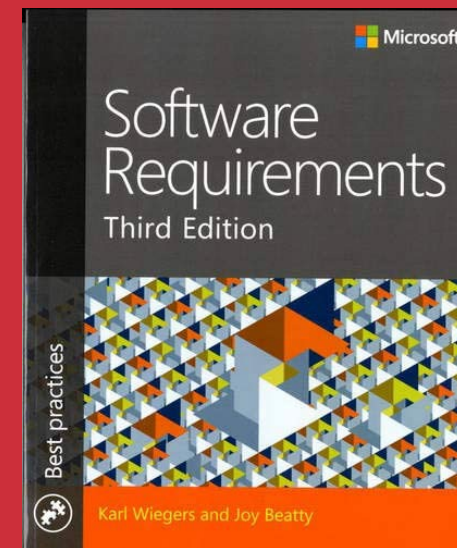
IMPORTANT! Changes may occur. Always check on Canvas and TimeEdit

Agenda for today's lecture

- Requirements Management
- Estimating (Chapter 9)
- Scheduling and Resourcing (Chapter 10)

Requirements Management

Based on notes by Prof. Knut H. Rolland
and the book Software Requirements 3rd Edition by
Karl Wiegiers and Joy Beatty



The hardest single part of building software is deciding precisely **what to build**. No other part of the conceptual work is as difficult as establishing the requirements. No other part of the work so cripples the resulting system if done wrong. No other part is **so difficult to rectify later**.

Brooks, Frederick, and H. Kugler. **No silver bullet** *Essence and Accidents of Software Engineering*. April, 1987.



How the customer explained it



How the Project Leader understood it



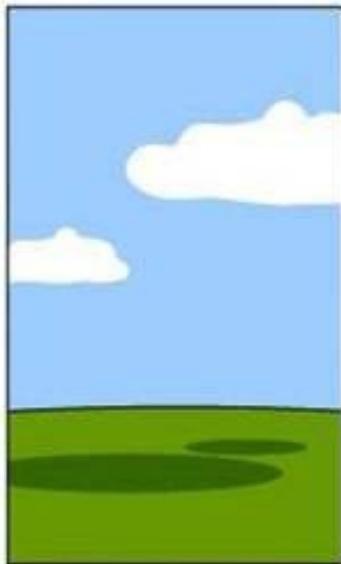
How the Analyst designed it



How the Programmer wrote it



How the Business Consultant described it



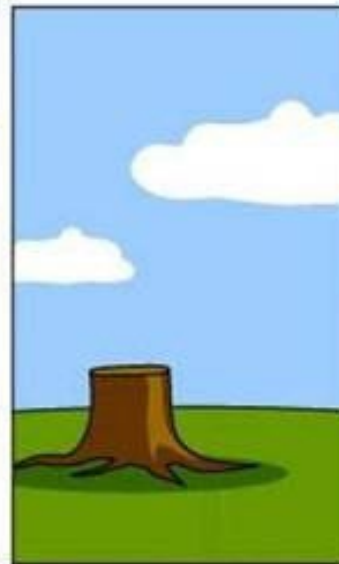
How the project was documented



What operations installed



How the customer was billed



How it was supported



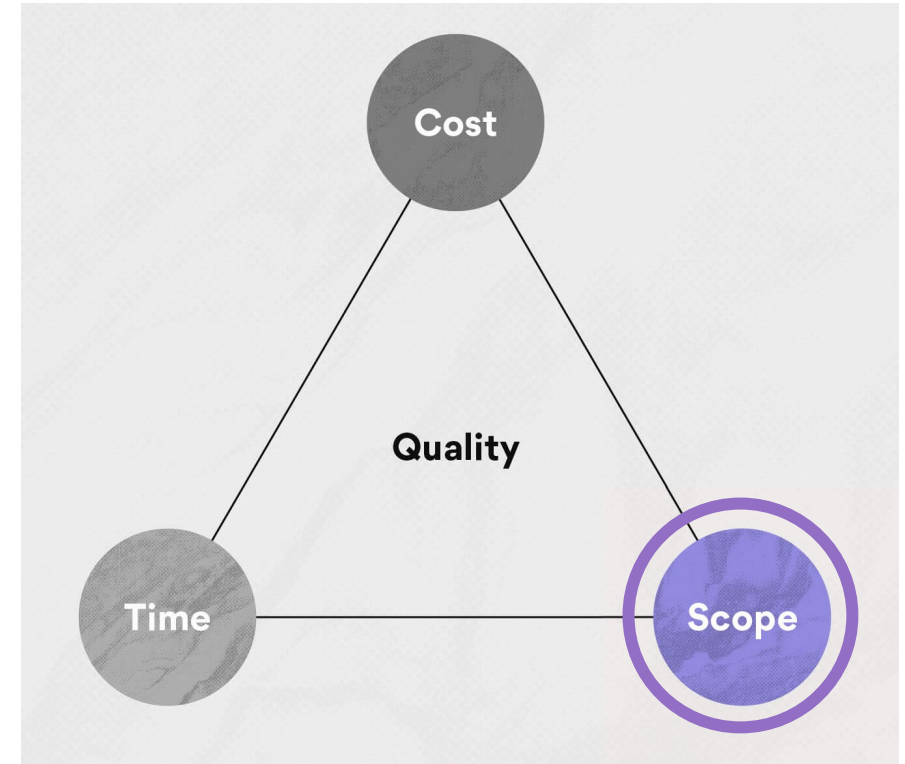
What the customer really needed

Users often find that they get something different from what they think they have ordered!

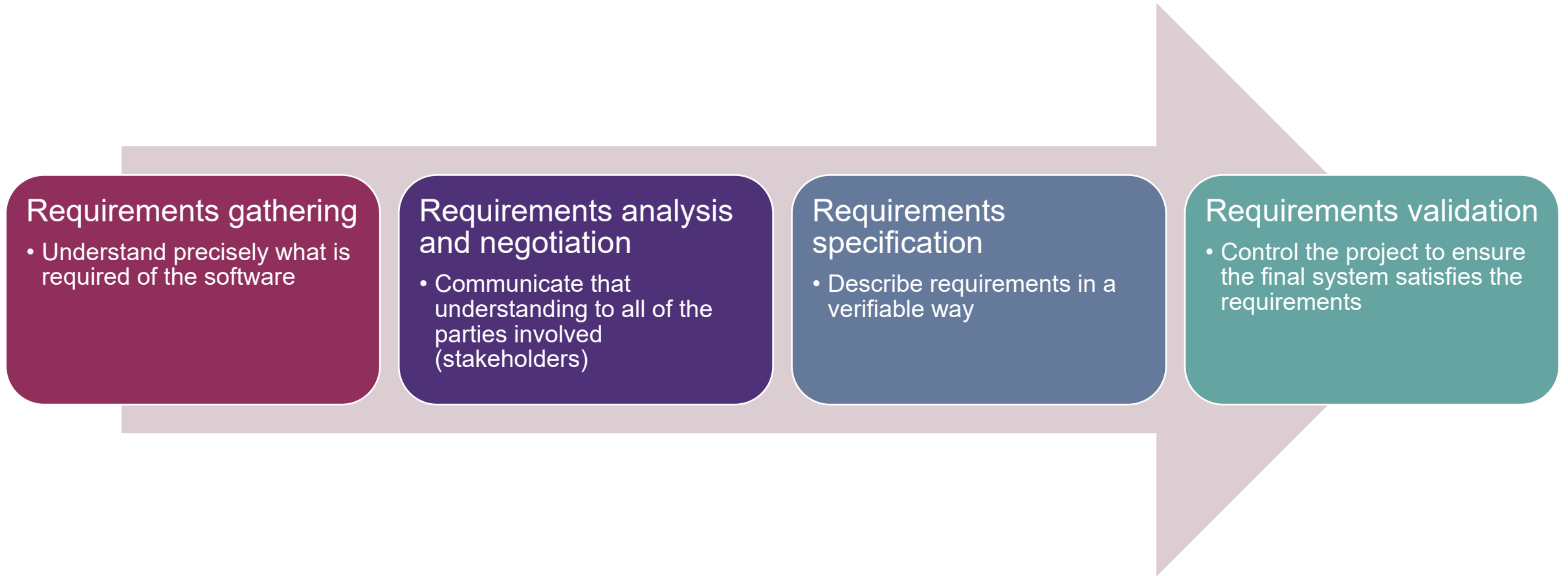
What is a “Software Requirement”?

A (nondefinitive) definition

- A description of something the software must do or property it must have
- The set of system requirements denote the problem to be solved and any constraints on the solution
 - Ideally, requirements specify precisely what the software must do without describing how to do it
 - Any system that meets requirements should be an acceptable

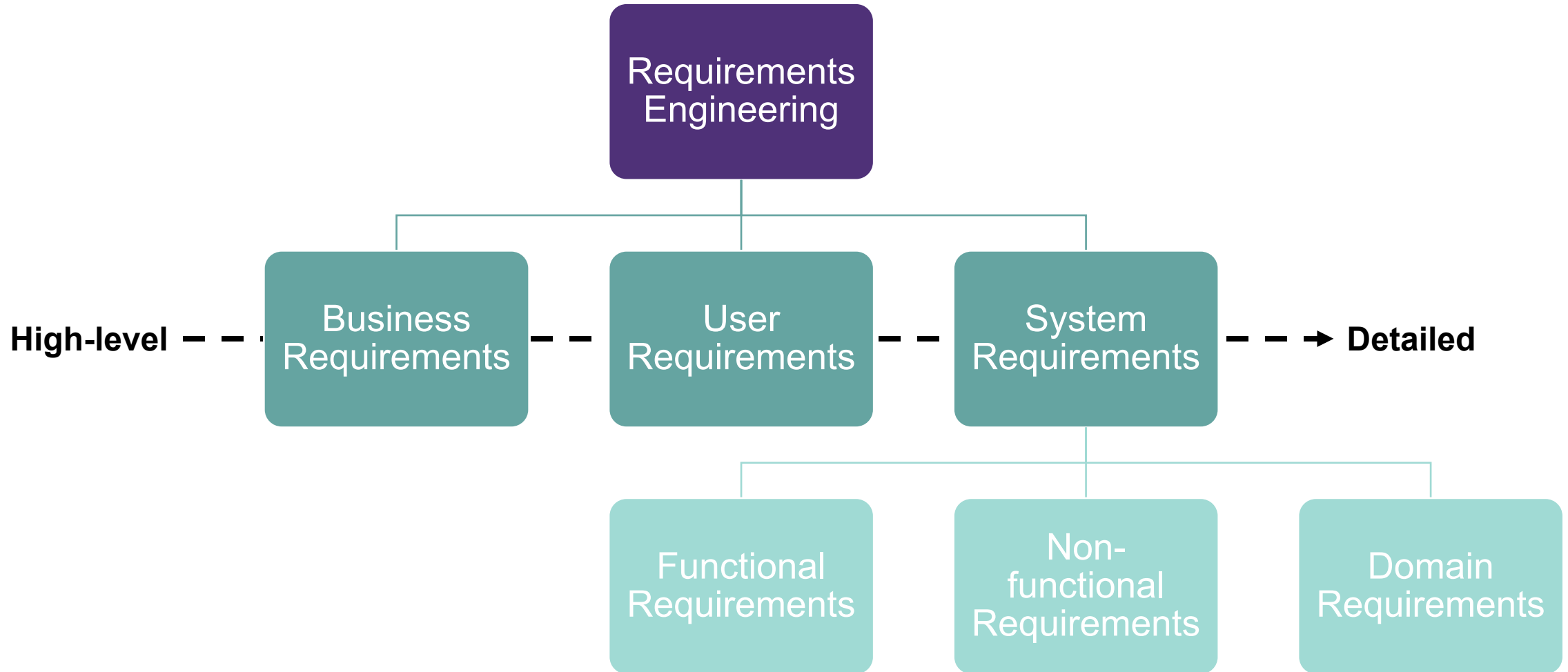


The Requirements Engineering Process (as envisioned)



Sommerville, Ian, and Pete Sawyer. **Requirements Engineering: a good practice guide**. John Wiley and Sons 113 (1997): 114.

Types of Requirements



Functional Requirements

describe “what” the product does

- Notice that some requirements “**must**” be delivered, while others “**should**” be delivered
- Also important to identify the requirements for traceability

Requirement ID	Requirement Statement	Must/Want	Comments
FR001	The website shall have a home page that lists the purpose of the organization.	Must	
FR002	The website home page shall list the next monthly meeting date, time and location.	Must	
FR003	The website shall have a page called “our members” where visitors can review our membership information by member name, business name, or business category.	Must	We have a spreadsheet that contains the member names, business names, contact information, etc.
FR004	Members shall be able to renew their membership via a credit card using the website, under a page called “member renewals”.	Must	Paypal
FR005	New members shall be able to sign up and pay for their membership via a credit card using a “join our organization” page.	Must	Paypal
FR006	A “Contact Us” page shall provide the names and contact information for every person on the executive.	Must	Name, role, email and phone.
FR007	A “Resources” page shall provide a list of links to organizations	Want	Town website, etc.

Non-Functional Requirements

describe “how” the product works

1.1 Non-Functional Requirements - Runtime Qualities		© www.businessanalysis.in
Performance	Group of requirements defining the required throughput and response time characteristics of the system.	
Throughput Requirements	The ability of the system to execute a given number of transactions within a given unit of time.	
Response Times	The allowable distributions of time which the system takes to respond to requests.	
Capacity	Group of requirements defining the capacity and scalability characteristics of the system.	
Capacity Requirements	Ability of the business system to ensure that the system continues to meet or exceed its service level agreements.	
Scalability	Ability to continue to function well as it (or its context) changes in size or volume in order to meet a user need.	
Availability (including Disaster Recovery)	The extent to which users are able to access and use the system in the expected times of service; and the extent to which the service must be resilient to component failures and disaster scenarios.	
Availability	The minimum proportion of time of given online service hours that the service should be available.	
Online Service Hours	Requirements stating when the service, or each aspect of the service, must be available to its users.	
Offline Requirements	The times available, if any, for non-online operations including batch processing and system maintenance activities.	
Resilience	The reliability and resilience characteristics of the system and/or its subcomponents.	
Recoverability	The ability to handle & recover from unexpected failures & outages of components both within & outside the system.	
Disaster Recovery	The ability to recover from & continue service following a major incident such as the loss of an entire data centre.	

Source: <https://www.businessanalysis.in>

Non-Functional Requirements

describe “how” the product works

1.1 Non-Functional Requirements - Runtime Qualities		© www.businessanalysis.in
Security	Required characteristics & mechanisms to provide system security against misuse/ malicious attack/ unauthorised access.	
Authentication	Correct identification of parties attempting access to systems, and protection of the systems from unauthorised parties.	
Authorisation	Mechanisms and structures required to authorise users to perform different functions within the systems.	
Data Confidentiality	Extent to which data needs to be protected and kept confidential.	
Compliance	The processes which will be used to confirm systems' compliance with an organisation's security standards & policies.	
Audit	Security driven auditing in order to ensure that the security guidelines and policies are not being breached.	
System Management	Collection of requirements pertaining to live systems management and the corresponding services.	
Alert Management	Requirements on the infrastructure and/or application to raise alerts in response to error or exception conditions.	
Service Level Reporting	Requirements for the capture of data for and production of reports on system performance against agreed service levels.	
System Monitoring & Measurement	The expected monitoring & measurement approaches to be taken for the live service infrastructure & applications.	
Maintenance and Administration	The adding and removing users and their privileges, monitoring the application and software distribution and installation.	
Usability and Accessibility	Qualitative properties of the final system relating to ease of use, user productivity, user contentment & accessibility support.	
Usability	The qualities of the system which affect user productivity and contentment with the use of the system.	
Accessibility	The ways in which the system will be accessible to users with particular or non-standard accessibility requirements.	
Audit Logging	Audit Logging requirements define a sequence of electronic entries that provide a retrievable record for an activity or action in the form of a report.	

Source: <https://www.businessanalysis.in>

Examples of non-functional requirement

- The system should respond to a search request within 3 seconds
- The system should be able to process a minimum of 10,000 users simultaneously
- The overnight update process must finish within 3 hours so data is available by 8 AM local time
- The system must meet Web Content Accessibility Guidelines WCAG 2.1
- Database security must meet HIPAA requirements
- Users shall be authenticated before loading a new page

Requirements Engineering

Sounds easy but hard to do in practice

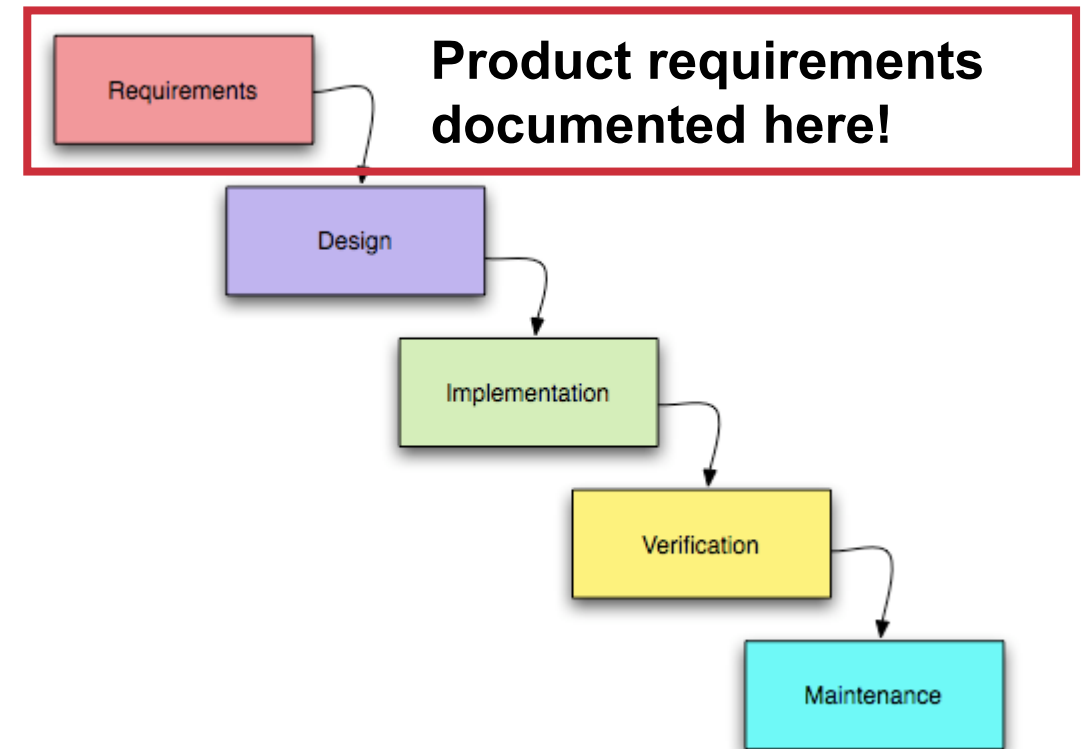
- **Comprehension** (understanding)
 - People don't (really) know what they want (...until they see it)
 - Superficial grasp is insufficient to build correct software
- **Communication**
 - People work best with regular structures, and software is complex and difficult to visualize
 - People confuse the different types of requirements, making the communication/documentation difficult
- **Control** (predictability, manageability)
 - Difficult to predict which requirements will be hard to meet
 - Requirements change all the time
 - Together can make planning unreliable, cost and schedule unpredictable
- **Joint Concerns**
 - Many issues cannot be solved individually (i.e., decisions about one necessarily impact another)
 - Difficult to apply "divide and conquer"

The traditional method

Assumptions:

- User needs are stable, accessible and can be documented
- User needs are easily 'gathered' through using various techniques and communicated to a broad audience
- Organizations are 'homogeneous' in terms of user needs, interpretations and views on ITs

Holmström, Jonny, and Steven Sawyer. **Requirements engineering blinders: exploring information systems developers' black-boxing of the emergent character of requirements.** European Journal of Information Systems 20.1 (2011): 34-47.

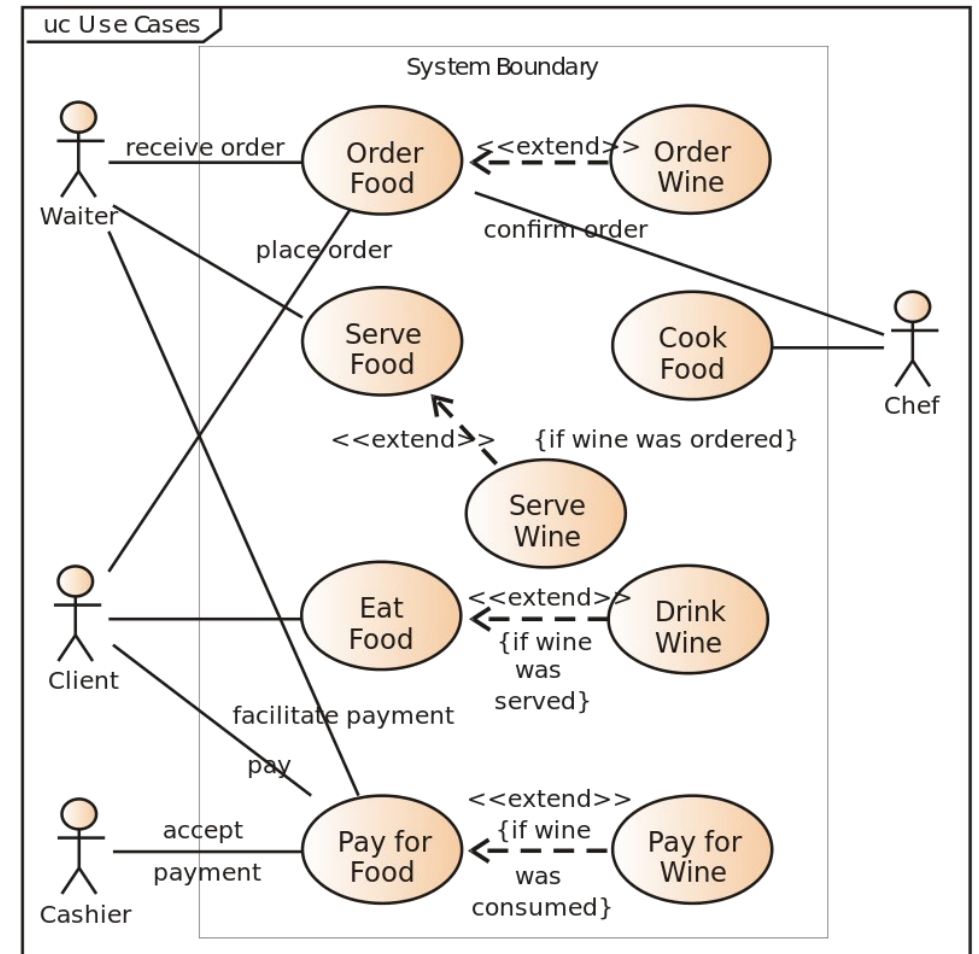


Use Cases

- A potential scenario in which a system receives an external request and responds to it
 - Simply, interactions between a system and a user (called actor)
- It is a technique to specify requirements using textual, structural and visual modelling

Jacobson, Ivar. **Object-oriented development in an industrial environment.** Conference proceedings on Object-oriented programming systems, languages and applications. 1987.

Jacobson, Ivar, Ian Spence, and Brian Kerr. **Use-case 2.0.** Communications of the ACM 59.5 (2016): 61-69.



Agile: a different way of manage requirements

- Focus on short-time planning
 - freeze requirements for a short time
 - flexibility in not defining everything 100%
- Self-managed and cross-disciplinary teams
 - Multiple competencies are needed for translating user needs into requirements
- Continuous learning throughout the process
 - Agile welcomes changing requirements, even late in development
 - Continuous interaction with customers will eventually 'reveal' the requirements
 - Negotiating – but this point is contested

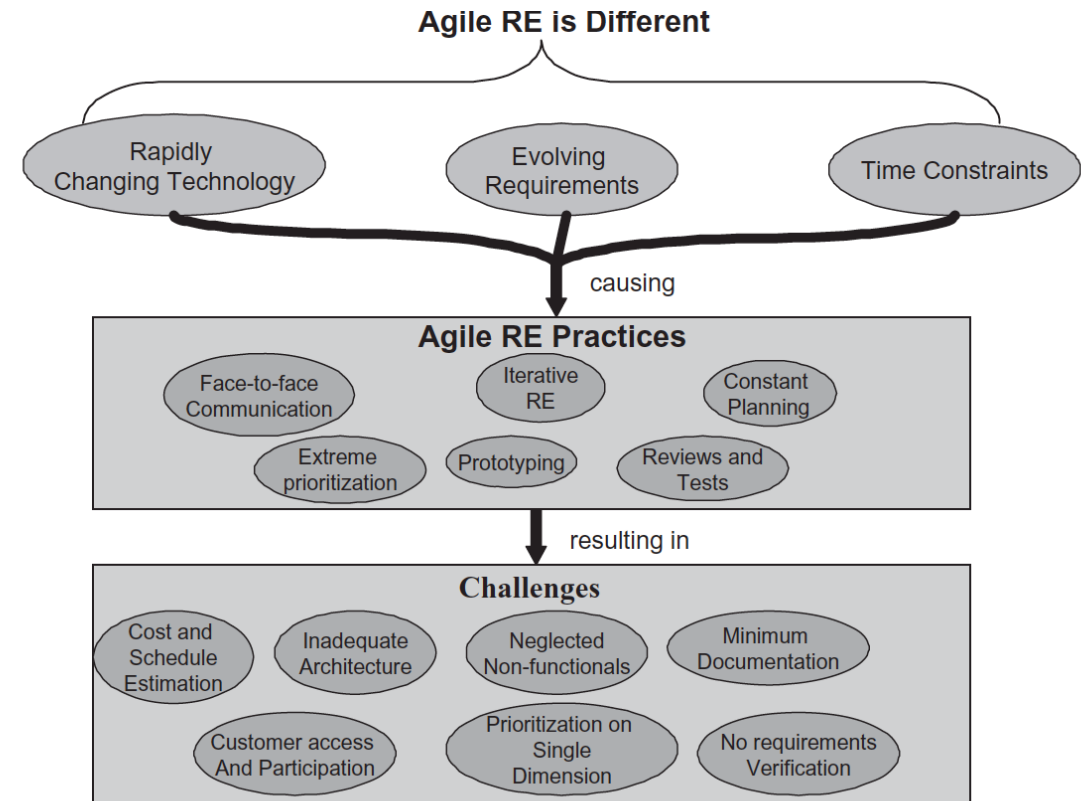
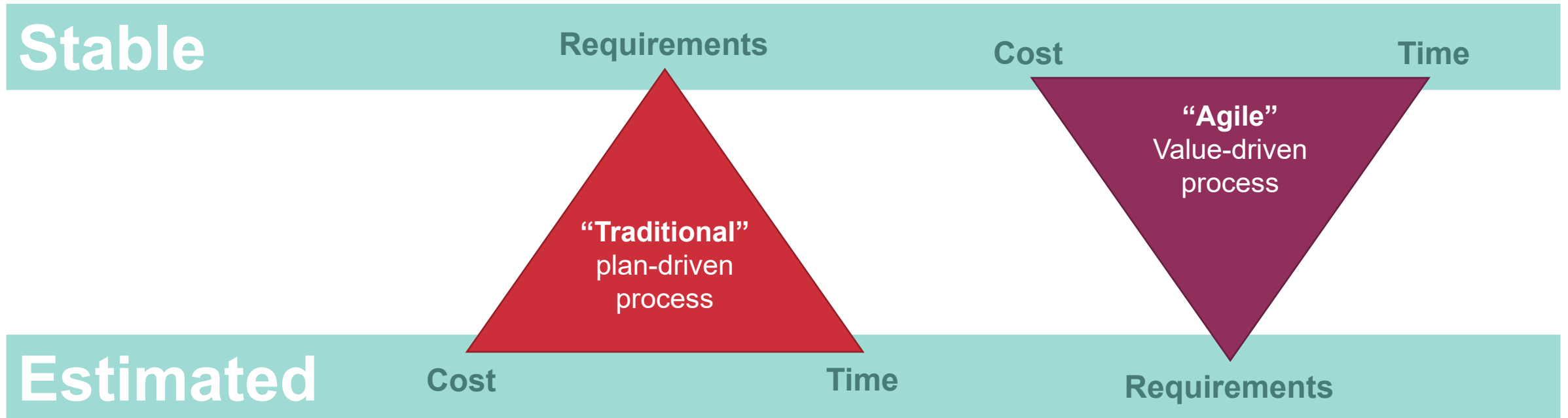


Figure 2. Agile RE practices and challenges.

Ramesh, Balasubramaniam, Lan Cao, and Richard Baskerville. **Agile requirements engineering practices and challenges: an empirical study.** Information Systems Journal 20.5 (2010): 449-480.

Consequence...

Goodbye iron triangle



Leffingwell, Dean. **Agile software requirements: lean requirements practices for teams, programs, and the enterprise.** Addison-Wesley Professional, 2010.

User story

A way to describe user needs

- A user story is a brief statement of intent that describes something the system needs to do for the user
- The story is the unit of functionality in an Agile project
 - Progress is measured by delivering tested, integrated code that implements a story
- A good user story should be
 - understandable to customers
 - valuable to the customer
 - small enough that the programmer can build half a dozen in an certain iteration
 - verifiable, i.e. developer-testable
- User stories need to be translated into more precise requirements
 - This is done as close to development as possible

User story

A way to describe user needs



As a <role>
I want <goal>
so that <benefit>

Acceptance criteria:

(Conditions of Satisfaction)

...

...

- As a **user**
- I want **to select a document**
- so that **I can share it with a friend**
- **Acceptance criteria:**
 - A friend should receive a link to the document
 - The user should be notified when the friend access the document

Chapter 9 – Project Planning: **Estimating**

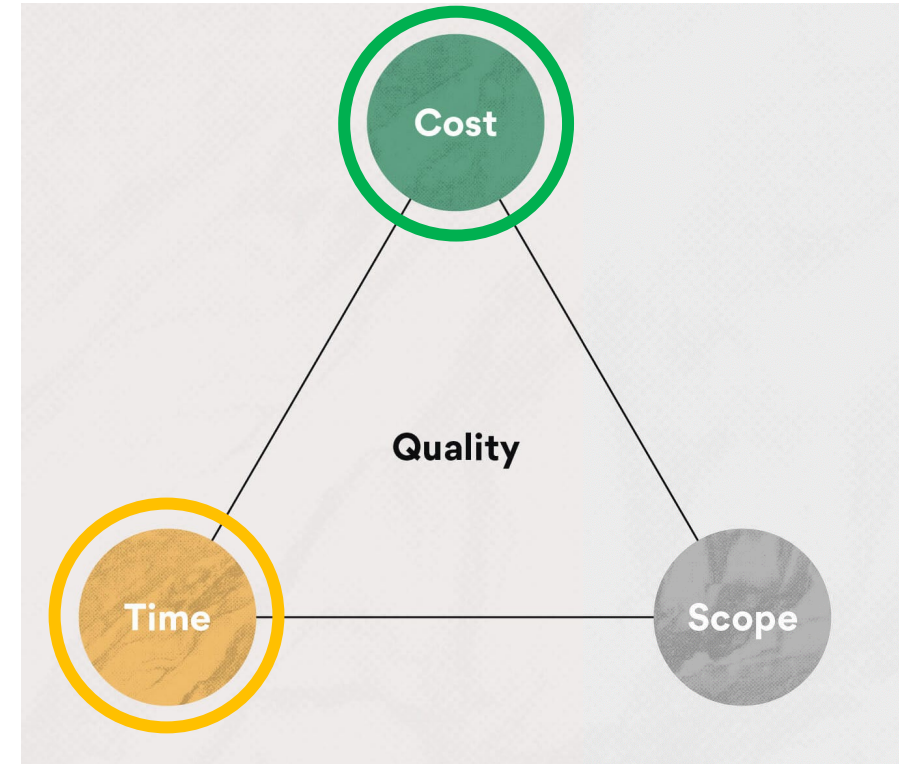
Cost and effort estimation

What is Estimation?

Definition

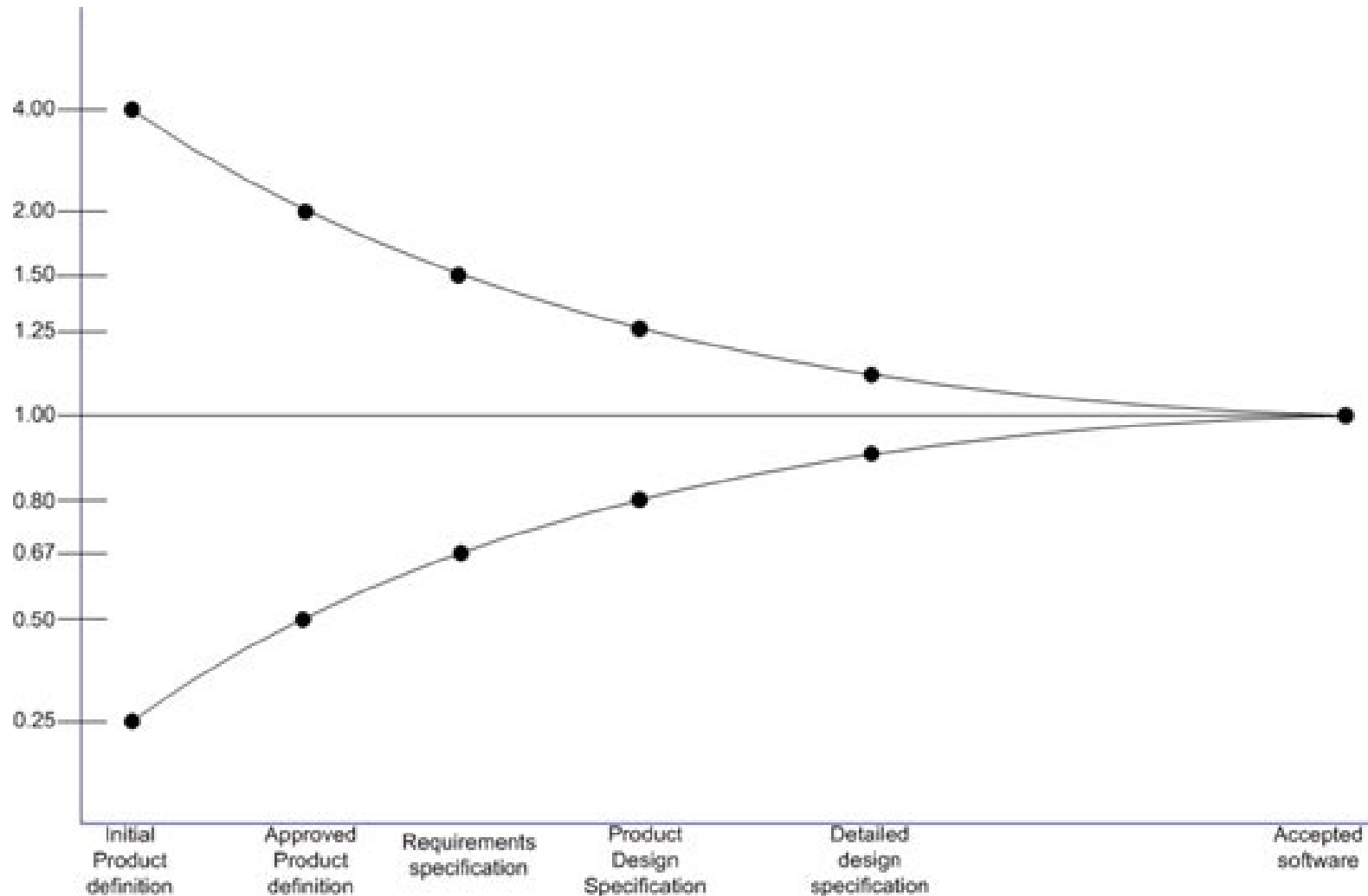
An assessment of the size of tasks with **costs, time spent** (hours and calendar time), and the number of people needed to realize the products in the project

The assessment is based on experience, various empirical and analyst techniques



Cone of Uncertainty

"Like shooting in the blind from a very long distance..."



WinWord for Windows 1 (McConnell, p 208)

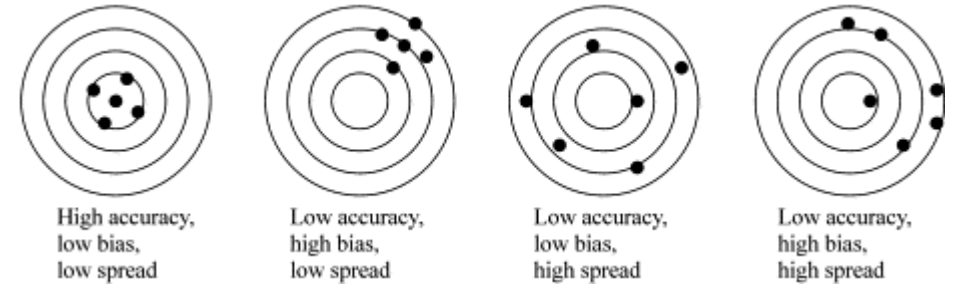
Example of over-optimistic estimation

Report date	Estimated days	Actual days
September '84	365	1887
June '85	395	1614
January '86	304	1400
June '86	334	1245
January '87	245	1035
June '87	152	884
January '88	122	670
June '88	153	518
August '88	123	457
October '88	120	396
January '89	92	304
June '89	92	153
July '89	92	123
August '89	92	92
November '89	0	0

McConnell, Steve. **Software estimation: demystifying the black art**. Microsoft press, 2006.

Why Estimation is Difficult?

- IS Projects tend to be one-off
 - They typically include innovative components for competitive advantage
- The initial estimates are often prepared long before there is a detailed specification of the requirement on which to base them
- IS Projects are seldom preformed by professional estimators
- Not enough accumulated experience or “blue books”



Jørgensen, Magne, and Dag I. K. Sjøeberg. **An effort prediction interval approach based on the empirical distribution of previous estimation accuracy.** Information and software Technology 45.3 (2003): 123-136.

- Estimation requires time and expertise
- Assessment of uncertainty should always keep up with the estimates
- Not a specific method that's always best
- One should almost always use several methods at once and compare
- One should build up a separate database of historical data
- Even if you have statistics from your own projects, this cannot be used mechanically to estimate new projects and completely different contexts

Practical Recommendations

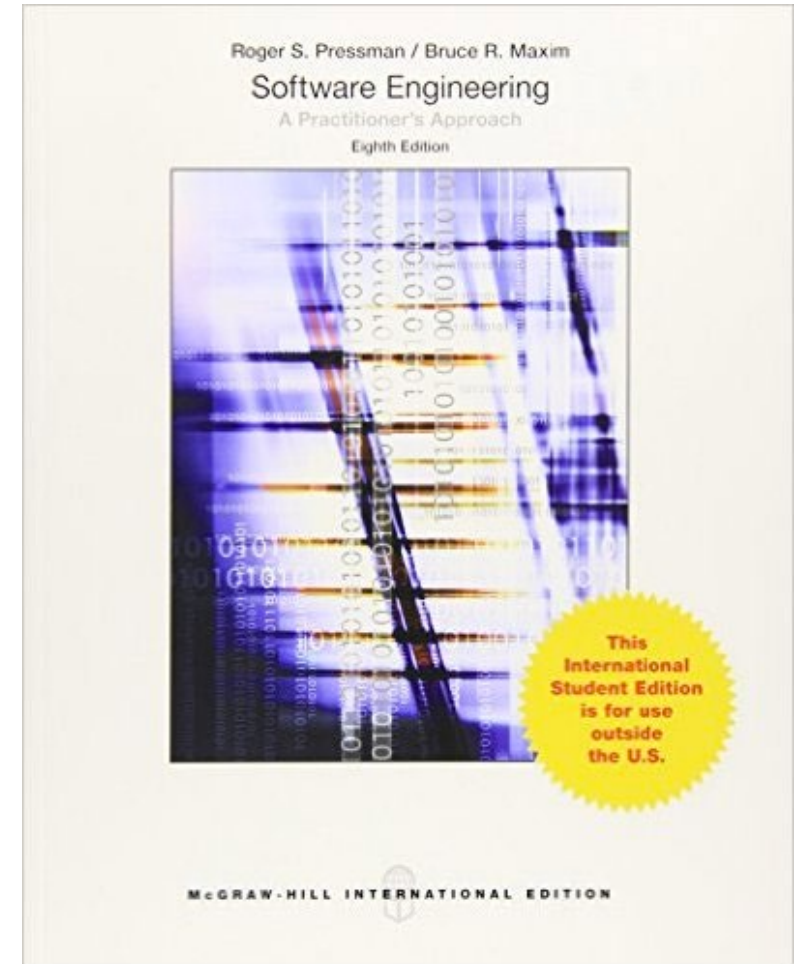
Also, what we learned from other Engineering Domains

- Data-driven
- Sensitivity analysis
- Multiple techniques
- Group estimates
- Training and Reflection
- Estimation and confidence
- Identify the known components and base estimates around them
- Firm estimates are based of firm specifications
- Measure progressively to improve our estimating accuracy

Estimation can never guarantee correct results

“Software cost and effort estimation will never be an exact science. Too many variables – human, technical, environmental, political – can affect the ultimate cost of software and effort applied to develop it.”

Pressman, Roger S. **Software engineering: a practitioner's approach**. Palgrave macmillan, 2005.



Techniques overview

two (not so different) views

Cadle and Yeates (2004)

1. Analogy method
2. Estimate based on scope of analysis phase
3. Estimate based on the scope of coding
4. Live estimation based on WBS/PBS
5. Delphi method
6. CoCoMo
7. Function Points
8. PERT estimating

Ruhe and Wohlin (2014)

1. Jorgensens simple approach
2. Critical path analysis with PERT, also known as 3-point estimation
3. Estimates based on WBS
4. CoCoMo
5. Estimation by analogy (EBA)
6. Mean magnitude of relative error (MMRE)

1. Estimation by Analogy (EBA)

old but reliable

- Estimates based on similar types of projects:
 - Type of organization
 - Size and complexity
 - The scope and functionality
 - Technically comparable; platform, programming language, tools etc.
- Easy to estimate if you carry out many similar projects with the same team, development environment and type of customers
- A major disadvantage is that one can overfocus on what is equal between different projects and underestimate what is different

2. Analysis Effort Method

Estimate based on the scope of the analysis phase



1. How much work is there to analyze the need and define the requirements for all the desired functions?

2. Assessing based on 3 factors:
S – max number of people working on the project
F – how familiar the work is, and
C – the complexity of the project

4. Calculates the size of all phases based on the requirement phase.

3. Determines the relationship between the different phases based on historical data and the assessment of S, F, and C

2. Analysis Effort Method

Three factors/sizes

		F	Description		
S	Description	1	All factors are known to the participants of the project. The project typically resembles a previous project	C	Description
1	IT project with 1 participant	2	The type of solution and techniques are known, but technical parts of the solution and/or development environment are unknown	1	Familiar algorithms, simple data structures, few files, few interactions
2	Small project with up to 4 participants	3	Unusual type of solution and specialist techniques not known to the participants, but technical parts of the solution known	2	One of the parts above not the case
3	Medium-sized project with up to 12 participants	4	Solution type or techniques new for developers but common otherwise, technical parts of the solution known	3	Two or more of the above factors are not the case
4	Large project with up to 30 participants	5	Solution type or techniques new for developers but common otherwise, technical parts of the solution also unknown	4	Major technical solution challenges
5	Very large project with more than 30 participants	6	An innovation project with high uncertainty	5	Complexity cannot be defined at this time
6	Size cannot be considered at this time	9	Not possible to assess at this time		

2. Analysis Effort Method

Example – Calculating factors

- We will assess how much work it is to create a new internet publishing solution using EPiServer for a company that wants to sell its products online
- In a pre-project, we have concluded the following:
 - **S = 3**: The project involves at most 6 people: two developers, two designers, a test manager and a solution architect. We do not count project manager and training activities. Only the work that will produce the product
 - **F = 2**: The project is planned to be developed with a team that doesn't know much about EPiServer already, but they are familiar with publishing solutions in general and .NET on which EPiServer is based
 - **C = 3**: The project on EPiServer will be integrated with the current customer register and a payment system. This entails greater complexity than usual
- Estimated number of hours for demand analysis: 500 hours

2. Analysis Effort Method

Example - Estimates

- According to the table of ratios for medium projects:
 - Analysis: 500 hours, makes up 5%
 - Design: 1500 hours, makes up 15%
 - Development: 5100 hours, amounts to 51%
 - Testing: 2900 hours, makes up 29%
 - **Total hours = 10.000**
- If each consultant has an average price of NOK 1075 per hour, this will cost **NOK 10.750.000**
 - In addition, comes project management, training, etc
- The estimate is high, but not unreasonable since there are several integrations

Table 9.1 Part of a table of ratios for the analysis and programming effort methods

<i>F</i>	<i>C</i>	<i>Analysis</i> %	<i>Design</i> %	<i>CUT</i> %	<i>Testing</i> %
1	1	2	14	59	25
1	2	2	14	55	29
1	3	5	14	47	34
1	4	7	16	43	34
2	1	2	15	63	20
2	2	2	14	59	25
2	3	5	15	51	29
2	4	7	16	47	29
3	1	5	16	59	20
3	2	5	16	55	24

Cadle, James, and Donald Yeates, eds. **Project management for information systems**. Pearson education, 2004. (p. 144)

2. Analysis Effort Method

Pros and cons

- Good technique if you have good historical data to build on – maybe you should build your own tables for the type of projects and platforms
- Cadle and Yeates (2008) have a relatively small share in analysis and design
- Especially good technique for early project phase or as a basis for offers to customer
- Disadvantage that one must have experience with the type of solution, platform and development environment, otherwise the ratios between the different phases will not be realistic
- Logic seems better approached to a waterfall model with phases than agile development

3. The Programming Method

Estimate based on scope of development job

- Based on the scope of the programming job itself and assesses the other activities based on this
- Can be based on requirement specification and/or some overall design
- Uses historical data from relevant projects
 - Can work well if you have good historical data and no alternative ways of estimating early in the project
- The simplest way of assessing the programs is to decide if each is likely to be small, medium or large
- Not always the amount of code that defines complexity and workload
 - Additionally, the estimator can include a measure of complexity (simple, average, complex)

3. The Programming Method

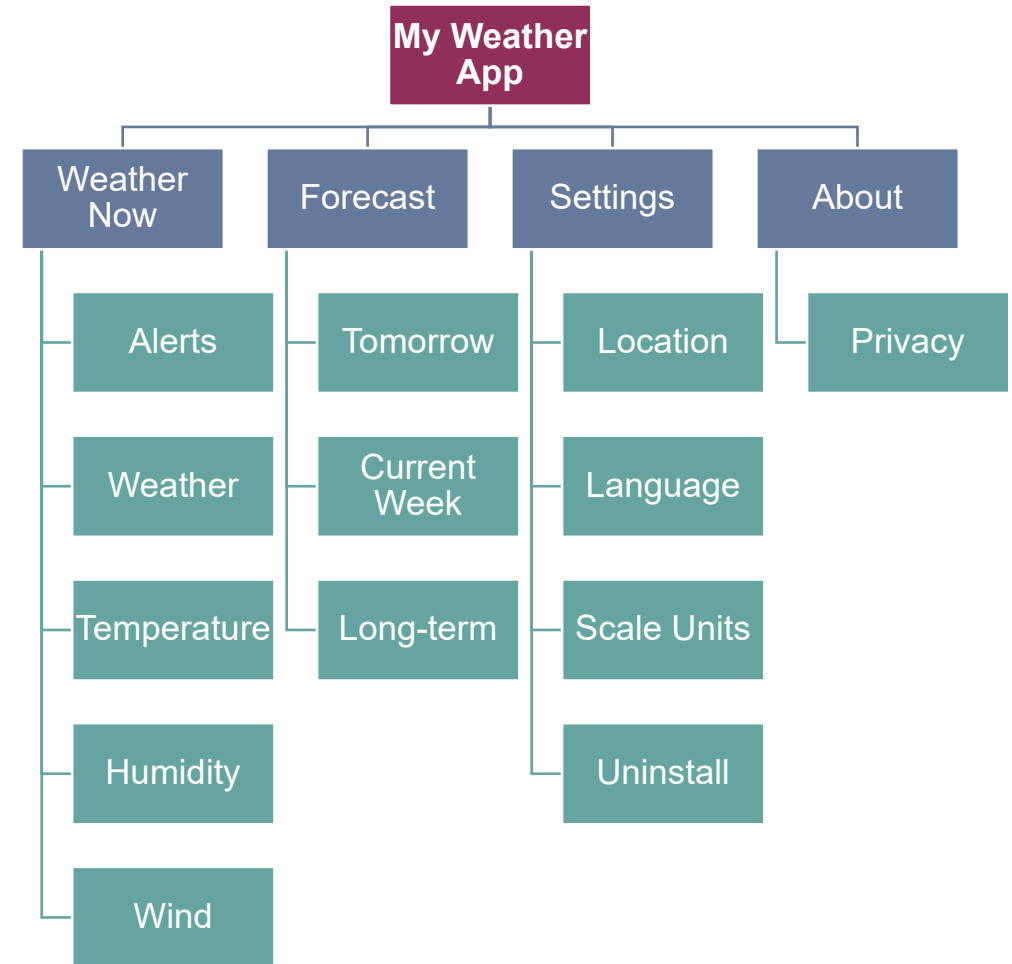
Estimate based on scope of development job

1. The simplest way is to decide if the task likely to be small, medium or large
2. The estimator uses historical data from other projects to establish a baseline for the programming task
 - Ex. Simple 5 days, medium 10 days, large 15 days
3. And then derive estimates for other activities using a benchmark
 - Not always the amount of code (size) alone defines workload
 - The estimator can include a measure of complexity (simple, average, complex)

Size	Complexity		
	Simple	Average	Complex
Small	5	5	10
Medium	5	10	15
Large	10	15	20

4. Direct/live estimation based on project breakdown

- Detailed estimates based on WBS or PWS
- Good if you have enough time, resources and detailed knowledge about what to deliver
- Requires a lot of work and expertise
- Usually used in developing plans for the immediate stage or sub-stage of a project
 - More approximate methods are used for the later stages until sufficiently detailed information comes to hand



5. Delphi Method aka Expert Opinion



Planning Poker

[sidenote]

Instructions

- The Product Owner provides a short overview of one user story
 - The team asks questions
- Each individual lays a card face down representing their estimate for the story
- Everyone calls their cards simultaneously by turning them over
- People with high estimates and low estimates should justify their estimate
 - and then the discussion continues
- Repeat the estimation process until a consensus is reached



Story Points Estimation

[sidenote]

How much we know	Dependencies	How much work effort	Story Points
Everything	None	< 2 hours	1
Almost everything	Almost none	Half a day	2
Something	Some	Up to 2 days	3
Almost nothing	Few	Few days	5
Nothing	More than a few	Around a week	8 – must be split into smaller itens
Nothing	Unknown	More than a week	13 – must be split into smaller itens

Pick a row that represents your task best; if it fits more than one, pick the higher one.

6. CoCoMo method

- Constructive Cost Model (CoCoMo) developed by Barry Boehm
 - Available in slightly different varieties, also evolved into CoCoMo2
- Developed a set of formulas to calculate man-months and time specified months a project should take based on the number source code lines
 - Difficult to use because the number of source code lines is hard to predict and varies widely
 - Measured in KDSI – delivered source instructions (thousands)
- Often used to assess the length of an IT project
 - Can also be used to double-check other estimates

Boehm, Barry. **Software engineering economics**. New York 197 (1981).

Boehm, Barry, et al. **Cost models for future software life cycle processes: COCOMO 2.0**. Annals of software engineering 1.1 (1995): 57-94.

6. CoCoMo Basic

Formula and example

- MM (approximately 152 working hours) = $2.4 (\text{KDSI})^{1.05}$
 - $2.4 \times (10,000)^{1.05} = \mathbf{26.92 \text{ man-months}}$
- TDEV (TDEV is the total development time) = $2.5 (\text{MM})^{0.33}$
 - $2.5 \times (26.92)^{0.33} = \mathbf{8.94 \text{ months}}$
- Approx. Number of persons = MM / TDEV
 - $26.92 / 8.94 = \mathbf{3 \text{ persons}}$

7. Function point analysis (FPA)

- **Function point** is a measure of business functionality a system provides to a user
 - Alternative measure to lines of code (LOC)
- FPA estimate effort by quantifying the functionality
 - Based on logical design of the software
 - Independent of technology
 - Across all projects and organizations
- It is a 3-step approach that requires a series of calculations
- It has been heavily criticized for not providing a better estimation than simpler techniques

Function Point Estimation Worksheet						
	Complexity					
	Simple		Average		Complex	
Description	Estimate	*	Estimate	*	Estimate	*
Outputs		4	3	5	30	7
Inputs	45	3	5	4		6
Inquiry Outputs		4		5	15	7
Inquiry Inputs		3		4	12	6
Files	20	7	51	10		15
Interfaces		5		7	9	10
Total Function Points (FP):						1297
Environmental Factor						Rating
Data Communications						5
Distributed Computing						5
Performance Requirements						3
Constrained Configuration						2
Transaction Rate						3
Online Inquiry and/or Entry						5
End-User Efficiency						4
Online Update						5
Complex Processing						5
Reusability						1
Ease of Conversion/Install						1
Ease of Operation						5
Used at Multiple-Sites						5
Potential for Future Change						2
Total Environmental Factor (N):						51
Complexity Adjustment Factor (CAF):						1.16
Adjusted Function Points (AFP):						1504.52

Use Case Points and Story Points

[sidenote]

- Use cases or stories as a starting point (see requirements, above)
- Good if already using UC or Stories
- Often suggested for large scale agile projects
- Similar to FPA, it requires a series of calculations

Armentrout, Alan W., and Regina M. Trujillo. **Function Points, Use Case Points, Story Points: Observations From a Case Study.** *CrossTalk* (2013): 23.

Actors analysis				
Complexity	Type	Weight	Quantity	Total weigh
Simple	Another system through batch files	1	1	1
Average	Another system through a protocol/API Any type of physical devices	2	2	4
Complex	A person through a graphical user interface	3	2	6
TOTAL			5	11

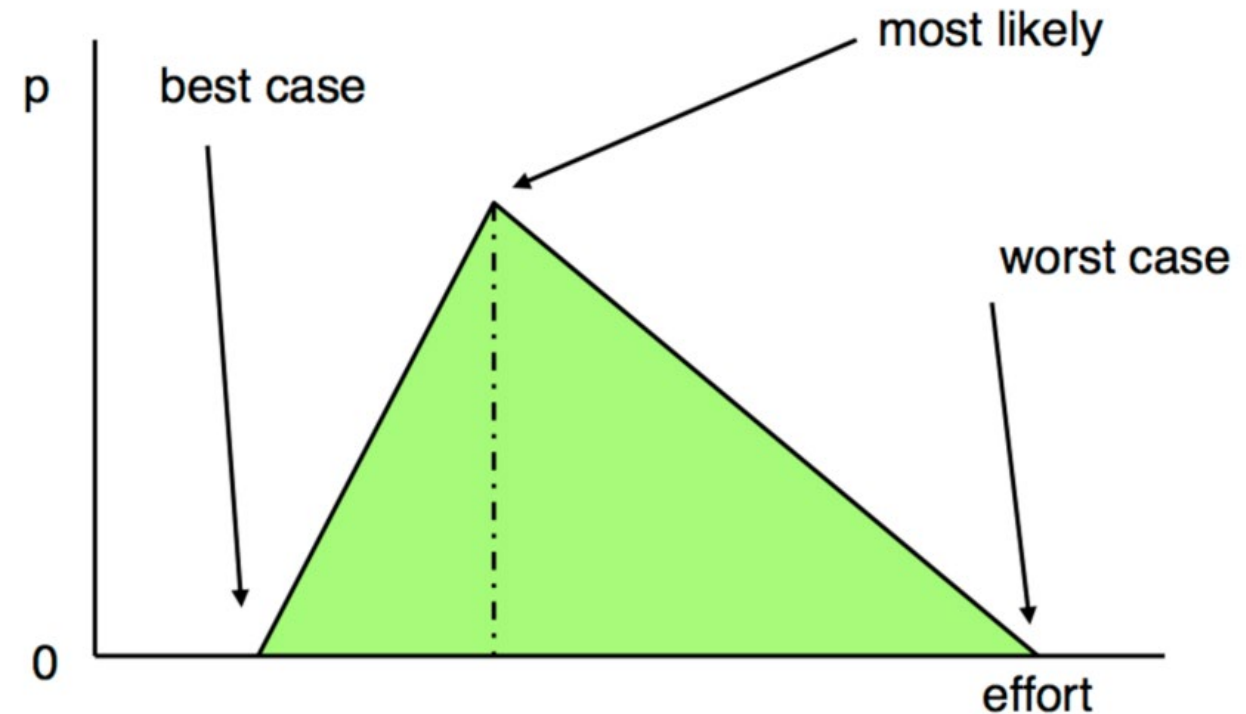
Use Cases analysis				
Complexity	Number of transactions	Weight	Quantity	Total weigh
Simple	3 or fewer	3	3	9
Average	4 to 7	6	2	12
Complex	More than 7	9	1	9
TOTAL			6	30
Total Unadjusted Use Case Points (UUCP)				41

8. PERT Estimating

3-point estimation

- Using the formula $(A + 4B + C)/6$, where
 - **A** is the most optimistic estimate
 - **B** is the most likely estimate
 - **C** is the most pessimistic estimate
- These values are calculated with the PERT chart using the latest start time (LST) and latest finish time (LFT)

Willis, R. J. **Critical path analysis and resource constrained project scheduling—theory and practice**. European Journal of Operational Research 21.2 (1985): 149-155.



Human factors

One of the reasons for wrong estimates

- People do not like to estimate complex things
 - Especially if they are accountable for that
- Contract negotiations and the desire to get the estimate can push up unrealistic estimates
- IT projects with great prestige and risk can have a particularly large impact on estimates
- **Expert judgement** are informal approaches for estimation based on intuition
- They work surprisingly well compared to more formal techniques used by untrained estimators

Jørgensen, Magne, and Dag I. K. Sjøeberg. **An effort prediction interval approach based on the empirical distribution of previous estimation accuracy.** Information and software Technology 45.3 (2003): 123-136.

Additional activities

Another reason for wrong estimation

- Many required activities are not part of estimation
 - Project management
 - Administrative tasks such as timekeeping and booking of meeting rooms
 - Presentations and review of documentation etc.
 - Quality assurance activities
 - User involvement
 - Team management
 - Coordination
- The larger the project, the more additional tasks

Tips:

- Add a percentage add-in for each task
- Explicitly estimate the additional activities
- Relate the activity to the project pipeline – for example, project management is done 4 hours a day throughout the project
- Relate the task to a role: project manager 50%, quality manager 20%

Questions?

Jefferson Seide Molléri
jefferson.molleri@Kristiania.no

Exercise

Based on the PID you delivered in the previous assignment, detail a project plan, including:

- Breaking down the project into work packages (WP)
- Estimate the effort/cost for each WP
- Assign each task to resources
- Link project resources to schedule and budget
- Create a timeline (or a Gantt chart) to monitor the project's progress

Bibliography

- Cadle, James, and Donald Yeates, eds. **Project management for information systems**. Pearson education, 2004.
- Ruhe, Günther, and Claes Wohlin, eds. **Software project management in a changing world**. Vol. 21. Berlin/Heidelberg, Germany: Springer, 2014.
- Wiegers, Karl, and Joy Beatty. **Software requirements**. Pearson Education, 2013.
- Sommerville, Ian, and Pete Sawyer. **Viewpoints: principles, problems and a practical approach to requirements engineering**. Annals of software engineering 3.1 (1997): 101-130.
- Pressman, Roger S. **Software engineering: a practitioner's approach**. Palgrave macmillan, 2005.
- Brooks, Frederick, and H. Kugler. **No silver bullet Essence and Accidents of Software Engineering**. April, 1987.

Next Lecture

A keynote with Wenche Eidjord

Wenche Eidjord is Senior Project Manager at Avinor and has for the last 16 years been managing complex IT implementation projects.

She is a computer science engineer with more than 30 years of experience within the IT business, primarily system development and project management, she has been working in industries such as finance, oil and gas, and transportation. For almost 10 years she was working for Oracle primarily as a consultant.

From 2012 to 2016 she was a program manager for implementing a multi airport operational system (for Avinor's 43 airports). This includes the procurement of the system, establishing a high availability infrastructure, design, build and test of the system, deployment and rolling out to the airports. From 2019 she has been the project manager introducing the new generation multi airport operational system planned to go live spring 2023.

<https://www.linkedin.com/in/wenche-eidjord-1374552>